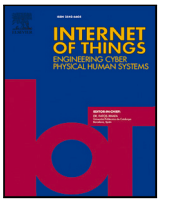
Internet of Things 22 (2023) 100699

Contents lists available at ScienceDirect

Internet of Things

journal homepage: www.elsevier.com/locate/iot



Research article

A deep learning approach for intrusion detection in Internet of Things using focal loss function

Ayesha S. Dina, A.B. Siddique, D. Manivannan *

University of Kentucky, Department of Computer Science, Lexington, KY, 40508, USA

ARTICLE INFO

Keywords:
Internet of Things
Intrusion detection
Cyber security
Data imbalance problem
Loss functions
Deep learning

ABSTRACT

Internet of Things (IoT) is likely to revolutionize healthcare, energy, education, transportation, manufacturing, military, agriculture, and other industries. However, for the successful deployment of IoT in various industries, methods for detecting and preventing security breaches need to be designed and implemented. Over the past decade, a number of researchers in academia and industry have used Machine Learning (ML) techniques to design and implement Intrusion Detection Systems (IDSes) for computer networks in general; however, not much work has been done for intrusion detection in IoT. Datasets collected by various organizations were used by many of these researchers to train ML models for predicting intrusions. It is common for datasets used in such systems to be imbalanced (i.e., not all classes have equal number of samples). Predictive models developed using machine learning algorithms can produce unsatisfactory results if imbalanced datasets are used for training the models. To remedy this problem, techniques such as random oversampling and undersampling do not produce robust models. Furthermore, ML models trained using traditional loss functions, such as cross-entropy loss, fail to accurately predict minority class instances. To overcome the data imbalance problem for intrusion detection in IoT, we leverage the specialized loss function, called focal loss, that automatically down-weighs easy examples and focuses on the hard negatives by facilitating dynamically scaled-gradient updates for training effective ML models. We implemented our approach using two well-known Deep Learning (DL) neural network architectures. We conducted extensive experimental evaluations using three datasets from diverse IoT domains and compared our proposed approach with state-of-the-art intrusion detection models. We found that, our approach (training DL models using focal loss function) performed better with respect to accuracy, precision, F_1 score and MCC score by as much as **24%**, **39%**, **39%**, and **60%**, respectively, compared to training them on the same datasets using cross-entropy loss function. We also compare our approach with two other state-of-the-art approaches and show that our approach performs better than these state-of-the-art approaches.

1. Introduction

Internet of Things (IoT), a network of interconnected devices which communicate with each other through wired and wireless networks, is likely to revolutionize healthcare, energy, education, transportation, manufacturing, military, agriculture, and other industries. IoT is already playing an important role in Intelligent Transportation Systems (ITS) in dealing with fleet management,

* Corresponding author.
E-mail addresses: adi252@uky.edu (A.S. Dina), siddique@cs.uky.edu (A.B. Siddique), mani@cs.uky.edu (D. Manivannan).
URLs: https://ayeshasdina.github.io/ (A.S. Dina), https://www.cs.uky.edu/~siddique/ (A.B. Siddique), http://www.cs.uky.edu/~manivann (D. Manivannan).

https://doi.org/10.1016/j.iot.2023.100699
Received 2 November 2022; Received in revised form 13 January 2023; Accepted 14 January 2023
Available online 16 January 2023

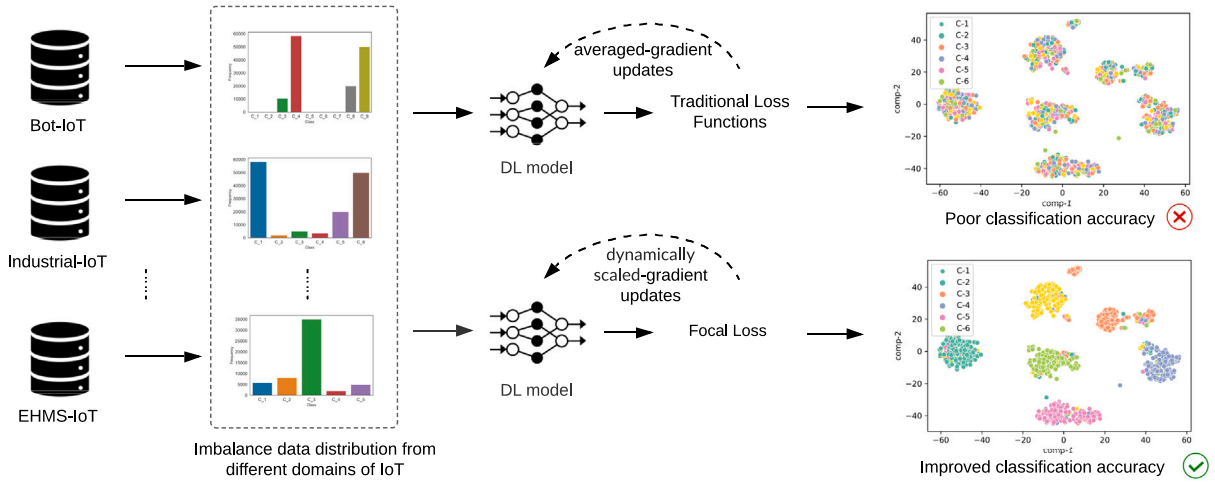


Fig. 1. Deep learning-based models that use traditional loss functions (e.g., cross-entropy loss) exhibit significantly poor performance for class-imbalanced classification tasks. In this work, we leverage focal loss function to train deep learning-based models to overcome the class imbalance challenge by enabling dynamically scaled-gradient updates to focus on the hard misclassified data points and reducing the loss for the well-classified ones.

traffic management, and driver assistance [1], among others. In order to remain competitive, many organizations have adopted IoT for collecting and processing data to guide their decision making process.

In 2021, there were more than 10 billion active IoT devices. According to Cisco [2], the number of IoT devices is likely to exceed 500 billion by 2030. Given the enormous interest of customers in connected devices such as cellphones, thermostats, door bell cameras, etc., IoT devices are likely to play an important role not only in industry but also in everyone's life. However, as with any other computer system, IoT are susceptible to cyber attacks. For example, on Friday, October 21, 2016 a series of Distributed Denial of Service (DDoS) attacks exploiting the vulnerabilities of IoT devices have been launched across the United States of America with Mirai malware [3]. So, protecting the devices in IoT which could be collecting and processing sensitive private data, from various cyber attacks becomes a necessity.

Machine Learning (ML), a widely used technology in many scientific applications, is playing an important role not only in securing IoT but also in various other aspects of IoT such as coordinating the wireless devices for efficient spectrum [4] usage in a distributed way. This is primarily because the problems that arise in wireless communication systems require classification, detection and optimization, for which ML algorithms are very suitable. It is noteworthy to mention that an attacker can also use ML to launch sophisticated attacks on IoT and cause significant damage to the IoT. So, ML is a double-edged sword — it can used to protect IoT and can also be used to launch attacks on IoT. Following ML approaches have been studied by various researchers for intrusion detection in computer networks: Neural Networks, Association Rules and Fuzzy Association Rules, Bayesian Networks, Clustering, Decision Trees, Evolutionary Computation, Hidden Markov Models, Inductive Learning, Naïve Bayes, Sequential Pattern Mining, Support Vector Machine, and Deep Learning (DL). Recently, DL-based models have demonstrated state-of-the-art results for classification task using a wide range of datasets [5].

A number of imbalanced datasets (i.e., datasets in which not all classes have equal number of samples) have been used for studying intrusion detection. A majority class in a dataset is a class with large number of samples, whereas the minority classes are the ones with small number of samples. Ratio of the number of samples in a minority class to the number of samples in a majority class may be as small as 1:100, as large as 1:1000, or even larger. Fig. 1 shows how imbalance in dataset can affect the performance of classifiers that use traditional loss functions (e.g., cross-entropy loss function) during training. Most researchers have either ignored the issue or attempted to balance data by oversampling (i.e., by randomly replicating the samples in minority classes) or undersampling (i.e., by randomly removing majority class samples). Over-sampling and under-sampling of data help in balancing datasets. However, oversampling may result in overfitting since the new samples are exact replicas of the original samples. On the other hand, under-sampling may result in a dataset that is too simple, as there are too few data samples to build an effective model, resulting in the underfitting problem. Generally, an overfit model has low bias and high variance, while an underfit model has high bias and low variance. Moreover, traditional loss functions, such as cross-entropy loss, perform averaged-gradient updates while training DL models, and consequently minority class instances are not properly attended to. Balancing data by adding synthetic data (e.g., data generated using Conditional Generative Adversarial Network (CTGAN) [6,7]) to minority classes, is effective as long as the synthetic data generated has high fidelity to the minority classes, which can be challenging; moreover, such approaches also require a long training time and high computation power because the datasets could become very large. Next, we explain how we address this issue which is the main focus of the paper.

1.1. Contribution of the paper

Focal loss function [8] has been used in computer vision to address the extreme imbalance between foreground and background classes during the training. To our knowledge, focal loss function has not been used for dealing with imbalanced data in datasets related to intrusion detection. Motivated by this gap, in this work, we leverage the focal loss function to train DL-based models to overcome the data imbalance issue. Considering that the focal loss function handles imbalanced datasets robustly, we employ it for the task of intrusion detection in IoT. In contrast to the traditional cross-entropy loss function that allows for averaged-gradient updates, the focal loss function facilitates optimization of the model by enabling dynamically scaled-gradient updates leading to down-weighing easy instances and forcing the model to focus on the hard misclassified examples. We implement focal loss function in two well-studied Deep Learning (DL) neural network architectures: (i) Feed Forward Neural Networks (FNNs) and (ii) Convolutional Neural Networks (CNNs). To evaluate the efficacy of using focal loss function in these DL-based models, we use datasets from three diverse domains of IoT. Specifically, we used Bot-IoT [9] from IoT sensors domains, WUSTL-IIoT-2021 [10] from industrial IoT domain, and WUSTL-EHMS-2020 [11] from healthcare monitoring domain.

We conduct extensive experimental analysis using these two DL-based models and compare their performance with several strong baseline models using original datasets as well as balanced datasets (i.e., balanced with oversampling as well as synthetic data samples generated using CTGAN [6,7]), and also employing traditional cross-entropy loss function and dice loss function [12,13]. We evaluate both binary and multi class classification tasks. Moreover, we also compare the performance of our approach with two recently proposed state-of-the-art IDSes – CNN-BiLSTM [14], and PB-DID [15] – on the three datasets. Our experimental results show that the two DL-based classifiers using focal loss function outperform all the baseline approaches as well as state-of-the-art models consistently on all three datasets with respect to F_1-score and MCC score. Specifically, our evaluation shows that our approach (CNN-focal, CNN trained using focal loss function) performed better with respect to accuracy, precision, F_1 score and MCC score by as much as **24%**, **39%**, **39%**, and **60%**, respectively, compared to CNN trained on the original datasets using traditional cross-entropy loss function.

Following is a summary of our contribution in this work:

- We investigate intrusion detection task in diverse domains of IoT – a highly critical but under explored area. Moreover, we study both multi-class and binary classification tasks for intrusion detection in IoT.
- We demonstrate that, we can effectively train DL-based models using focal loss function to overcome imbalanced data in IoT datasets.
- To our knowledge, no one has used focal loss function to deal with data imbalance issue in datasets related to IoT for intrusion detection.
- Our thorough experimental analysis shows that our proposed approach outperforms a wide range of strong baseline models as well as state-of-the-art models on three datasets from diverse IoT domains.

Rest of the paper is organized as follows: In Section 2, we discuss the related works. In Section 3, we provide details of our proposed approach. In Section 4, we discuss the experimental setup and performance evaluation results are presented in Section 5. Section 6 concludes the paper.

2. Related works

In this section, we discuss some of the recent research works that are related to security of IoT. This section will also serve as a short survey of recent research work done on security of IoT.

2.1. Evaluation of various ML models

Many researchers have evaluated the performance of various ML models. The anomaly based intrusion detection system for IoT, proposed by Ullah et al. [16], is modeled based on CNN. Gad et al. [17] used ToN_IoT dataset designed by Moustafa [18] to evaluate the performance of various ML models. Liu et al. [19] detect anomalies in IoT Network Intrusion dataset by applying various ML algorithms. To speed up intrusion detection, Branitskiy et al. [20] use distributed data processing system. Shafiq et al. [21] design a new feature selection algorithm to filter features that suit the selected ML algorithm accurately. Churcher et al. [22] compare the performance of various ML models on Bot-IoT dataset, developed by Moustafa et al. [9] . Disha et al. [23] evaluated the performance of various ML classifiers using UNSW-NB15 [24] and ToN_IoT datasets. Hasan et al. [25] analyze the performance of various ML algorithms in predicting attacks on IoT sensors in IoT systems. Verma et al. [26] assess the performance of various ML classifiers using the datasets CIDDS-001 [27], UNSW-NB15, and NSL-KDD [28], as well as using Friedman and Nemenyi tests.

2.2. Machine learning frameworks

Several researchers [29–33] proposed ML frameworks for solving various security related issues in IoT. Yang et al. [29] develop an ML framework for intelligent IoT networks. The framework leveraging Software Defined Network (SDN) and Network Function Virtualization, developed by Bagaa et al. [30], deals with various threats in IoT. Arachchige et al. [31] present a framework, called PriModChain, for ensuring privacy of Industrial Internet of Things (IIoT) data. Liu et al. [32] present a malicious node detection framework for handling a specific type of insider attack in IoT, called conditional packet manipulation attack. Makkar et al. [33], propose an ML framework for detecting spam in IoT networks.

2.3. Using ML for solving miscellaneous problems in IoT

Roy et al. [34] propose a machine learning-based two-layer hierarchical intrusion detection mechanism for effectively detecting intrusions in IoT networks while meeting the resource constraints of the IoT. In particular, this model is capable of utilizing the resources in the fog layer of an IoT network by deploying multi-layered feedforward neural networks in the fog-cloud infrastructure for the purpose of detecting network attacks. By incorporating a fog layer into the equation, real-time analytics of traffic data can be performed closer to IoT devices and end users since analysis is dynamically distributed across the fog and cloud layers. Liang et al. [35] point out that ML algorithms can be useful in detecting intrusions while at the same time they can be vulnerable to attacks at various stages of their implementation; they also discuss how ML algorithms can be used in launching sophisticated cyber-attacks. Sun et al. [36] use ML to model attacks using botnets. Amouri et al. [37] present an IDS for mobile IoT. Sivananthan et al. [38] use a combination of SDN and ML techniques to manage IoT devices. Zheng et al. [39] discuss the issues related to applying the privacy-preserving ML methods developed for cloud computing systems in the context of IoT.

Saha et al. [40] present a technique for detecting unknown system vulnerabilities in IoT. Guerra et al. [41] observe that, since the types and behavior of attackers change over time, network traffic collected and labeled becomes obsolete to capture new types of attacks. Using the Principle Component Analysis (PCA) method, Wahab et al. [42] study the change in the variance of the features across the intrusion detection data streams in IoT and present an online deep neural network that dynamically adjusts the sizes of the hidden layers to cope with these changes. Khan et al. [43] point out that the existing ML models used in the cyber-security area follow the black-box model and developed a method to solve this problem.

Ferrag et al. [44] compare the performance of centralized and federated deep learning with three of the well known deep learning approaches using three datasets. Zolanvari et al. [10,45] acknowledge the importance of ML and big data analytics for analyzing and securing IoT as well as securing IIoT. They built a real-world testbed to conduct cyber-attacks and designed an IDS to demonstrate how an anomaly detection system based on ML algorithms can perform well in detecting backdoor, command injection, and SQL injection attacks. Moustafa [18] presents a testbed architecture which allows the creation of dynamic testbed networks, which in turn allows the interaction of edge, fog and cloud tiers of an IoT network. They deployed this architecture by executing real-world normal and attack scenarios and collected the labeled dataset, called ToN_IoT.

Li et al. [4] present an ML based framework for automated decision making for spectrum sharing in next-generation regulatory spectrum management. Yacchirema et al. [46] propose an ML-based system for detecting falls of elderly people. Rehm et al. [47] develop a clinical decision support system for managing patients in intensive care unit. Meidan et al. [48] present an ML based method for detecting unauthorized IoT device-types whereas Salman et al. [49] present an ML framework for identifying device-type as well as malicious traffic in IoT. Liu et al. [50] present a survey of ML methods used for identifying IoT devices as well as detecting compromised IoT devices. Ma et al. [51] propose an ML-based method for evaluating the trustworthiness of IoT devices.

Zeeshan et al. proposed a Protocol-Based Deep Intrusion Detection (PB-DID) architecture, in which the authors compared features from UNSW-NB15 and Bot-IoT data sets using flow and Transmission Control Protocol (TCP) [15]. Initially, finding similarities between the UNSW-NB15 dataset and the Bot-IoT dataset, they combined both datasets by considering features that fall into both the flow and TCP categories in order to address the imbalance issue in the dataset. Following that, they were classified into three categories: normal, DoS attacks, and DDoS attacks using a deep neural network. As part of the preprocessing process, they combined training data with testing data, and then separated them into training data and testing data at random. We compared the performance of this model in our experiments.

As a result of combining 1D-CNN layers and Bi-LSTM layers, Sinha et al. proposed a hierarchical model [14]. Long short-term memory (LSTM), a subset of RNN, is used to identify long-term temporal patterns in a dataset, enabling the construction of a predictive model, whereas CNN is used to identify spatial features of a dataset. On two state-of-the-art datasets NSL-KDD and UNSW-NB15, the authors performed binary and multi-class classification. For the purpose of balancing the two datasets, they also used random oversampling. To start with, it should be noted that the training and testing sets were initially combined in both datasets. Afterwards, the training and testing sets were randomly divided. We compared the performance of this model in our experiments.

2.4. Intrusion Detection in Industrial Internet of Things (IIoT)

Industrial Internet of Things (IIoT) refers to interconnected sensors, and other devices networked together with other industrial applications such as manufacturing and energy management. IIoT helps in collecting and processing data from various points of interest in industry which helps the administrators in improving efficiency and productivity by refining and optimizing process controls. Several researchers [52–59] have focused their attention on developing IDSes for IIoT. The IDS developed by Beaver et al. [52] used several ML algorithms on a dataset consisting of labeled remote terminal unit (RTU) telemetry data from a gas pipeline system in Mississippi State University's Critical Infrastructure Protection Center. The IDS designed by Ullah and Mahmoud [55] also used the same dataset used in [52]. However, they used J48, a Decision Tree (DT) classifier and I Bayes classifier in their IDS. Keliris et al. [53] used Support Vector Machine (SVM) in their simulated testbed in MATLAB controlling Tennessee Eastman (TE) chemical process.

Eigner et al. [54] aimed at detecting man-in-the-middle attacks on a custom-built conveyor belt system using K - Nearest Neighbors (K-NN) classifier. He et al. [56] designed an IDS to detect false data injection attacks on smart grids. Potluri et al. [57] designed an IDS for industrial networked control systems using Support Vector Machine (SVM), and deep belief networks. Alves et al. [58] employed the k-means clustering algorithm for classification. Aboelwafa et al. [59] present a method using autoencoders (AEs) for detecting false sensor data injection attacks in IIoT. Chakraborty et al. [60] use actual IIoT sensor readings and build an

ML-based approach for detecting IIoT attacks in real time. Zhou et al. [61] identify the characteristics of a massive IIoT and present ML-based solutions that correspond to those characteristics and discuss their limitation. Gyamfi et al. [62] present a low overhead NIDS for resource-constrained devices in IIoT. Khan et al. [63] present an LSTM auto-encoder based IDS for IIoT to detect intrusions in real time. Telikani et al. [64] present a fog computing enabled framework, called DeepIDSFog, for IIoT. Koroniotis et al. [65] implemented an IIoT testbed that is suitable for IIoT architecture-enabled smart airports.

Many of the above mentioned works ignored the data imbalance issue while some of them addressed the issue by adding synthetic data or used random oversampling to balance the datasets. Moreover, many of them used datasets designed for general computer networks, not for IoT. In this paper, we used three unbalanced datasets from three different domains of IoT and studied the performance of two well known DL-models using focal loss function on these datasets and showed that they outperform the competing methods.

3. Proposed approach and classification methods used

First, we discuss how we formulate the task. We then describe how to overcome the challenge of class imbalance using the focal loss function. Finally, we elaborate on the deep learning-based models employed in this work along with the implementation details.

3.1. Task formulation

We model the input data as a two-dimensional matrix $X = (x_1, x_2, x_3, \dots, x_N)$, in which $x_i \in \mathbb{R}^D$ $(1 \le i \le N)$ is a vector with D dimensional feature space. The label y_i is associated with every x_i $(1 \le i \le N)$ and $\forall i$, $y_i \in \{1, \dots, L\}$. As a result, we have a dataset of N samples with L distinct attack categories. The feature vector $x_i \in \mathbb{R}^D$ is mapped to a label $y_i \in \{1, \dots, L\}$ using a learnable function $Y = f(x)$. An estimate of f is obtained using the labeled training dataset in supervised learning setting. We refer to this function as $\hat{f}(x)$. The objective is to make $\hat{f}(x)$ as close as possible to $f(x)$. In contrast to traditional classification tasks, where all the classes have roughly equal number of instances, the focus of this work is to overcome the challenge of class imbalance, i.e., few classes in the dataset constitute a major chunk of the dataset, whereas the remaining classes have only few instances – a phenomena that is prevalent in all the IoT datasets.

3.2. Focal loss function

Class imbalance is prevalent in numerous datasets in various scientific fields, including computer vision, biomedical informatics, and intrusion detection. The cross-entropy loss is a traditional loss function and is extensively used in the classification tasks. Let us consider cross-entropy loss for binary classification.

$$CE(p, y) = \begin{cases} -log(p), & \text{if } y = 1 \\ -log(1-p), & \text{otherwise} \end{cases} \tag{1}$$

where $y \in \{1, -1\}$ identifies the ground truth for negative and positive classes, respectively, $p \in [0, 1]$ is the estimated probability for the label $y = 1$. Essentially, it tries to reduce the average loss over the entire dataset using negative log likelihood. As a result, any deep learning-based model that uses such loss functions (e.g., cross-entropy loss) for training the model will be able to accurately predict the majority class, but will not be able to correctly predict the minority class, since the objective is to minimize the average loss over the training set.

Recently, specialized loss functions (e.g., dice loss and focal loss [8,12,13]) have been proposed to address the class imbalance issue in computer vision. For example, dice loss uses the dice coefficient to address the problem, as follows:

$$DSC = -\frac{2 * \sum_{1=1}^{N} y_i * g_i}{\sum_{i=1}^{N} y_i + \sum_{i=1}^{N} g_i} \tag{2}$$

where y_i and g_i are the ground truth and predicted values for ith sample, respectively, and N is the total number of samples. Nonetheless, in the case of highly class imbalanced data with small denominators such as intrusion detection task, the dice loss gradient is inherently unstable [66]. Consequently, misclassification of a few samples can result in a significant decrease in the dice coefficient, which is unfavorable for minority classes [67] and hence does not perform well for intrusion detection task.

To address this problem in intrusion detection task, we employ focal loss function [8] that had originally been introduced for object detection scenarios, wherein there is an extreme imbalance between foreground and background classes during training. Focal loss has been generally employed in object detection in computer vision, but we demonstrate that it is also applicable in intrusion detection task with imbalanced datasets due to its sparse-specific characteristics. In order to simplify notation, we can define p_t as follows:

$$p_t = \begin{cases} p, & \text{if } y = 1 \\ 1 - p, & \text{otherwise} \end{cases} \tag{3}$$

The key idea in focal loss function is to reshape the loss function so that easy examples are down-weighted and training is focused on hard negatives. To achieve this, a modulating factor $(1-p_t)^{\gamma}$ is added to the cross-entropy loss function with a focusing parameter $\gamma \ge 0$. The focal loss is defined as follows:

$$FL(p_t) = -(1 - p_t)^{\gamma} log(p_t) \tag{4}$$

In order to obtain a balanced variant of the focal loss, we can write it as follows:

$$FL(p_t) = -\alpha_t(1 - p_t)^{\gamma} log(p_t) \tag{5}$$

where a weighting factor α is used to solve the imbalanced issue in the dataset. Intuitively, in the case of a high γ, the minority classes will receive greater weight than the majority classes and vice versa.

Both α and γ are hyperparameters. In this work, we experiment with two of the most common neural network architectures: Feed-forward Neural Network (FNN) and Convolutional Neural Network (CNN) and employ focal loss function for training and demonstrate their efficacy on a wide range of IoT instruction detection datasets.

3.3. Deep learning-based models

This subsection discusses the two DL classification algorithms and how they are used to classify the input data with traditional loss functions and focal loss functions.

3.3.1. Feed-forward Neural Network (FNN)

It has been demonstrated that FNNs can be effectively used for pattern classification, clustering, regression, association, optimization, control, and forecasting [68–71]. There are three layers in FNN: an input layer, an output layer, and H number of hidden layers. Let us consider $W_h \in \mathbb{R}^{Q \times P}$, and $W_o \in \mathbb{R}^{N \times M}$, the weight matrices for hidden layer and output layer respectively where Q is number of input neurons, P is the number neurons in a hidden layer and M is the number of output neurons. There is a weight vector associated with each row of these matrices. A hidden layer's output matrix can now be expressed as follows:

$$H = f(XW_h + b_h), \tag{6}$$

where $X = \{x_1, x_2, x_3, \ldots, x_N\}$ represents the input matrix, b_h represents the bias matrix, and $f(.)$ represents the hidden layer's activation function.

The equation representing output layer can be expressed as follows:

$$\hat{Y} = g(HW_o + b_o) \tag{7}$$

Here, $g(.)$ is the activation function and b_o is the bias matrix of the output layer.

3.3.2. Convolutional Neural Network (CNN)

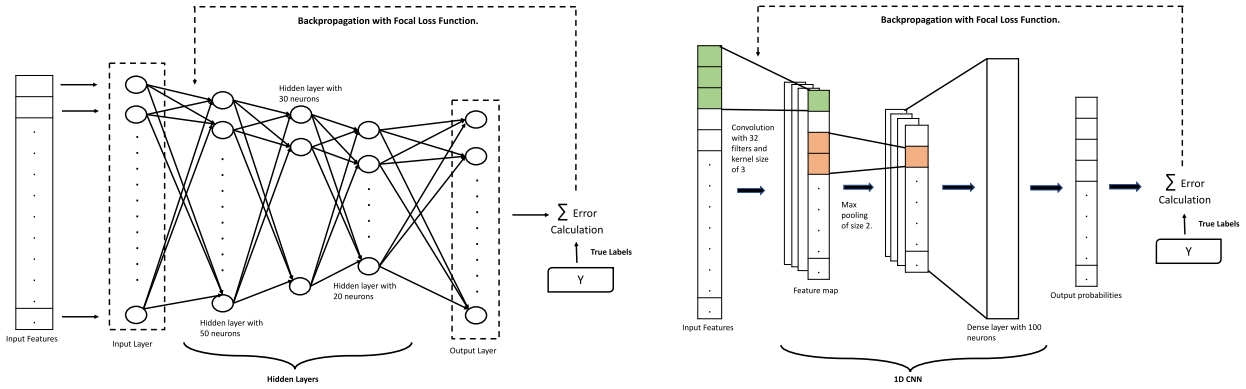
Performance of CNNs in many other fields is outstanding [72–75]. The input features of this neural network are transformed into meaningful information by convolutions, which are then used to construct the subsequent layers of neural network. A convolutional layer is usually a combined convolutional layer used for extracting features, and performing linear operations. A convolution is achieved by using multiple kernels or filters. Typically, a convolution can be described as:

$$\hat{Y} = x \times k + b \tag{8}$$

The kernel k is of dimension $n \times m$. x represents the input and b represents the bias.

3.4. Implementation details of deep learning models

We used the deep learning models FNN and CNN in our evaluation. The FNN we used consists of three hidden layers, each containing 50, 30, and 20 neurons; and the output layer contains neurons matching the number of attack types. For WUSTL-EHMS-2020, we use single hidden layer with 50 neurons. Finally, the final classification is performed using the softmax function. We use relu activation function, and Adam optimizer. Our dataset consists of a sequence of one-dimensional data; thus, we used a one-dimensional convolutional neural network (conv1D) for evaluation. Fig. 2(a) **is the architecture of the proposed FNN using focal loss.**. For conv1D, we used 32 filters with a kernel size of 3. Maxpooling has been used with a pool size of 2. Next, we used a dense hidden layer that consisted of 100 neurons, and finally we used a layer that consisted of neurons matching the number of attack types. We did not use the dense hidden layer with 100 neurons for the Bot-IoT dataset. Similar to FNN, for conv1D we applied relu activation function, and Adam optimizer. We trained each of these networks for 100 epochs with early stopping. Fig. 2(b) **illustrates the proposed CNN architecture using focal loss.** We used cross-entropy as the traditional loss function for both DL models in the baselines. We used the focal loss function as a specialized loss function for our implementation. The focal loss has two hyperparameters, namely, γ and α. While training on the Bot-IoT dataset: for FNN, we set α and γ as 2 and 1, respectively; for conv1D, we set these parameters as 5 and 5, respectively. While training on the WUSTL-IIoT-2021 dataset: for FNN we set γ and α as 2.5 and 0.15 respectively; and for conv1D, we set γ and α as 2.0 and 0.3 respectively. Finally, for training on the WUSTL-EHMS-2020 dataset: for FNN, we set γ and α as 2 and 2 respectively; and for conv1D, we set γ and α as 2 and 0.2 respectively. In order to implement and evaluate the neural networks, we used Tensorflow along with the Nvidia GPU driver version 455.32.00 and cuda version 11.1. All the related code is publicly available at: https://github.com/ayeshasdina/Intrusion-Detection-IoT.



(a) FNN (b) CNN

Fig. 2. Architecture of FNN and CNN.

4. Evaluation of proposed approach for intrusion detection in IoT

In this section, we discuss the evaluation criteria, datasets used for evaluation, and competing methods evaluated for comparison.

4.1. Evaluation criteria

We used the following quantitative metrics to evaluate the performance of various ML-classifiers: (i) Accuracy (**Acc**), (ii) Precision (**Pre**), (iii) Recall (**Rec**), and (iv) F_1**-Score** [76,77]. In our experiment, we computed the macro average for all four parameters, which is the desired metric for evaluating the methods for imbalanced datasets. **Acc** indicates how well the algorithm predicts the occurrence of a particular event. **Pre** refers to the frequency with which the algorithm predicts different types of intrusions. **Rec** indicates the proportion of actual intrusions that the algorithm predicted as intrusions. The reciprocal of the arithmetic mean of **Pre** and **Rec** is called the F_1**-Score** (i.e., the harmonic mean of **Pre** and **Rec**). Recently, Matthew's Correlation Coefficient (**MCC**) has been shown to produce more informative and truthful score than **accuracy** and F_1**-Score** [78] in evaluating classification performance. MCC produces a high score only if the prediction achieves good results in all of the four categories (True positives, False negatives, True negatives, and False positives), as can be seen from Table 1. Moreover, MCC is not affected by the imbalance of datasets. We also used MCC as one of the metrics in our evaluation.

Table 1 gives the formulas for calculating these metrics. We used these metrics to evaluate various machine learning models by counting the number of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). We evaluated both binary classification and multi-class classification tasks.

Precision is independent of accuracy. Accuracy measures how close is the result to the true or accepted value. On the other hand, precision refers to how close are the measurements of the same item to each other; in other words, precision is the percentage of relevant results. Recall refers to the percentage of relevant results that are correctly classified by the model. So, it is possible to be very precise but not very accurate; similarly, it is also possible to be accurate without being precise. F_1-score takes both precision and recall into account to measure the accuracy of the model. F_1-score tries to give more weight to false negatives and false positives. MCC produces high score only if the model achieved good results in all of the four categories (TP, TN, FP and FN) and hence is considered more reliable than other metrics.

4.2. Datasets

In this subsection, we discuss various datasets that we used for evaluating various approaches for intrusion detection in IoT.

4.2.1. Bot-IoT dataset

To generate Bot-IoT dataset [9], multiple tools were used to implement a variety of botnet scenarios, and packet capture files were organized into directories based on attack types. In addition, the authors used simulated Internet of Things sensors. They then used Correlation Coefficient and Joint Entropy techniques to statistically analyze the features of the dataset. In order to improve the performance of the machine learning models and to enhance the effectiveness of the prediction model, more features were extracted and added to the features set [21]. It is recommended that the extracted features be labeled for better performance, such as attack flow, categories, and sub-categories. There are four sub-components within the IoT testbed, which are simulation, networking platform, feature extraction, and forensics analytics. Along with data related to **Normal** traffic, this dataset includes data related to the following attack types: Denial of Service (**DoS**) attack, Distributed Denial of Service (**DDoS**) attack, **Reconnaissance** attack, and **Theft**. A total of 15 features are included. Bot-IoT dataset contains a training dataset as well as a testing dataset. Table 2 shows the distribution of various types of data items in the training and testing datasets of Bot-IoT. We can see from this table that the class types **Normal** and **Theft** have 296 (i.e., 0.01%) and 52 (i.e., 0.002%) training samples, respectively. In contrast, DDoS and DoS samples of the training dataset account for 53% and 45%, respectively.

Table 1
Performance metrics used in evaluating ML-models.

Metric	Formula for calculating the metric
Accuracy (Acc)	$\frac{TP+TN}{TP+TN+FP+FN}$
Precision (Pre)	$\frac{TP}{TP+FP}$
Recall (Rec)	$\frac{TP}{TP+FN}$
F_1 Score	$2 * \frac{Pre*Rec}{Pre+Rec}$
MCC Score	$\frac{TP*TN-FP*FN}{\sqrt{(TP+FP)*(TP+FN)*(TN+FP)*(TN+FN)}}$

Table 2
Data distribution in training and testing datasets of Bot-IoT dataset.

Class	Train	(%)	Test	(%)
DDoS	1233052	52.52	385309	52.51
DoS	1056118	44.98	330112	44.99
Reconnaissance	58335	2.48	18163	2.48
Normal	296	0.01	107	0.015
Theft	52	0.002	14	0.002

Table 3
Data distribution in WUSTL-IIoT-2021 training and testing datasets after we split.

Class	Train	(%)	Test	(%)
Normal	797261	92.71	221462	92.71
DoS	56379	6.56	15661	6.56
Reconnaissance	5932	0.69	1648	0.69
Command Injection	185	0.02	52	0.02
Backdoor	152	0.02	43	0.02

4.2.2. WUSTL-IIoT-2021 dataset

The dataset WUSTL-IIoT-2021 [10], contains data related to industrial Internet of Things (IIoT). This dataset was generated using a supervisory control and data acquisition architecture described in the IIoT testbed [45]. This testbed was designed to simulate real-world industrial systems as closely as possible, and to allow for actual cyber-attacks to be carried out. The authors have collected 2.7 GB of data in approximately 53 h. A preprocessing and cleansing process was carried out on the collected dataset (removing rows with missing values and corrupted values, i.e., invalid entries), as well as rows with extreme outliers. This dataset has 45 attributes. It contains data related to the following four distinct types of attacks: **DoS**, **Command Injection**, **Reconnaissance**, and **Backdoor**. *We would like to emphasize that this dataset does not have separate testing and training datasets. So, we sorted the dataset based on the timestamp of the data items, and then we took the first 80% of the samples for training and the rest for testing*. Table 3 shows the distribution of data related to various types of attacks in WUSTL-IIoT-2021 training and testing datasets, after this split. From Table 3, we can see that the Normal class type accounts for approximately 93% of the training dataset; on the other hand, data samples related to DoS, Reconnaissance, Command Injection, and Backdoor attacks accounts for only 7%, 7%, 0.02%, and 0.01% respectively.

4.2.3. WUSTL-EHMS-2020 dataset

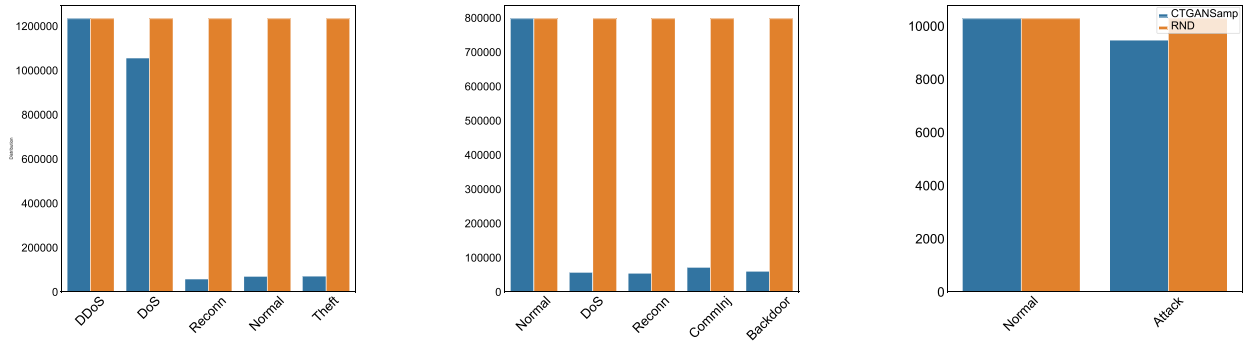
The dataset WUSTL-EHMS-2020 [11] was generated using a real-time Enhanced Healthcare Monitoring System (EHMS). The testbed collected both network flow metrics and patients' biometrics. There are four components to the EHMS testbed: medical sensors, gateways, networks, and control with visualization. In this testbed, the data flow begins with the sensors attached to the patent's body, and then continues to the gateway [11]. Using switches and routers, the gateway transmits the data to the server for visualization. The data can be intercepted by an attacker before it reaches the server. This dataset contains data about man-in-the-middle attacks, including spoofing and data injection. In this dataset, data items have 43 features: 35 of which are network flow metrics and eight are patients' biometric features. There are two types of labels for samples in this dataset: **Normal** and **Attack**. Based on the Source MAC address feature, the authors labeled the samples with the attacker laptop's MAC address as 1, while the rest were marked as 0. WUSTL-EHMS-2020 dataset is similar to WUSTL-IIoT-2021 dataset in that it does not have separate testing and training datasets. *Since it does not contain the timestamp feature, we split the dataset into training and testing datasets at random*. Table 4 shows the distribution of samples for the two types of attacks in training and testing datasets of WUSTL-EHMS-2020 after this split. As we can see, large portion of the samples are of **Normal** class type.

4.3. Competing models

In order to demonstrate the effectiveness of our proposed approach, we conducted extensive experiments on several other DL-based approaches. Additionally, we compared our approach with some of the recently proposed state-of-the-art machine learning models. Next, we describe the details of various models we trained for comparison.

Table 4
Data distribution in WUSTL-EHMS-2020 training and testing datasets after we split.

Class	Train	(%)	Test	(%)
Normal	10275	87.47	2855	87.44
Attack	1472	12.53	410	12.56



(a) Bot-IoT (b) WUSTL-IIoT-2021 (c) WUSTL-EHMS-2020

Fig. 3. Data distribution in the training sets of Bot-IoT, WuSt-IIoT-2021, and WuSt-EHMS-2020 after adding synthetic data using random oversampling and CTAGNSamp.

4.3.1. Baseline models

ORG: models were trained using original datasets (i.e., without balancing the dataset). We used the original training samples from the datasets for training the classifiers using traditional loss function, i.e., cross-entropy loss function.

RND: models were trained on the balanced datasets using random oversampling. We used random oversampling to balance the three training datasets. Random oversampling involves the duplication of samples in minority classes at random in order to balance the dataset. A representative sample of the population is selected from each subject independently during the entire sampling process [79]. After balancing training datasets with random oversampling: the training samples for each class type are approximately 1233052, 797261, and 10275 in Bot-IoT, WUSTL-IIoT-2021, and WUSTL-EHMS-2020, respectively. The orange bars in Fig. 3 shows the distribution of data for various attack types after balancing training datasets of Bot-IoT, WUSTL-IIoT-2021, and WUSTL-EMHS-2020 using random over sampling. We refer these models to as FNN-RND and CNN-RND in our experiments. We trained both models using cross-entropy loss function.

CTGANSamp: models were trained on the training datasets balanced using synthetic samples. We balanced the datasets with synthetic instances generated using CTGAN [6,7]. CTGAN uses a GAN-based (generative adversarial network) model to create synthetic tabular data from original tabular data. For Bot-IoT training dataset, the number of samples after adding synthetic data generated using CTGAN are 69215, and 71238 for the attack types Normal, and Theft, respectively. We decided not to add more synthetic samples to the DDoS, DoS, and Reconnaissance class types since the number of samples for those class types is already high. Similarly, after adding synthetic samples to WUSTL-IIoT-2021, the number of training samples is 54447 for Reconnaissance attack class, 70867 for Command Injection attack class, and 60231 for Backdoor attack class. The training dataset of WUSTL-IIoT-2021 contains large number of samples of types normal and DoS; so, no new samples were added to these attack classes. For WUSTL-EHMS-2020 training dataset, we added synthetic samples to only the attack class to increase the total number of samples in the attack class to 9472. The blue bars in Fig. 3 illustrate the distribution of data for various attack types in the training datasets of Bot-IoT, WUSTL-IIoT-2021, and WUSTL-EMHS-202020 after balancing them with synthetic data generated using CTGANSamp. We trained all the models on these datasets using cross-entropy loss function in our experiments. We name these models as FNN-CTGANSamp and CNN-CTGANSamp.

Dice: models were trained using dice loss function. For dealing with data imbalance issue in image segmentation, dice loss is usually used [12,13]. Dice loss is calculated based on the dice coefficient. In general, the dice coefficient can be calculated by dividing the sum of the ground truth and predicted values by two times the intersection of the ground truth and predicted values. We used dice loss function on all three original training datasets to train the DL models for both multi-class and binary class classification. We used dice loss as one of the competing methods for comparing performance in this work. For this, we used a smoothing parameter of $1e^{-7}$. We refer the models FNN and CNN using dice loss function as FNN-Dice and CNN-Dice in our experiments.

4.3.2. State-of-the-art models

In this subsection, we describe two state-of-the-art approaches recently proposed in the literature, which we compared with our approaches.

CNN-BiLSTM. CNN-BiLSTM [14] uses a 1D-CNN layer with activation function relu and a maximum pool size of five. In order to reduce training time, batch normalization is used. Bi-LSTM layers have been arranged throughout the model in a manner which

Table 5
Performance of the evaluated methods on Bot-IoT.

DL models	Classifier's name	Acc	Pre	Rec	F_1	MCC
State-of-the-art	CNN-BiLSTM	0.1407	0.2477	0.2563	0.0778	−0.0246
	PB-DID	0.5252	0.1717	0.2037	0.1448	0.0009
FNN	FNN-ORG	0.8834	0.5073	0.6345	0.5436	0.7868
	FNN-RND	0.8614	0.4990	0.4990	0.5275	0.7623
	FNN-CTGANSamp	0.8808	0.4991	**0.8652**	0.5540	0.7885
	FNN-Dice	0.4499	0.0900	0.2000	0.1241	0.0000
	FNN-Focal (This work)	**0.9155**	**0.5559**	0.6380	**0.5784**	**0.8825**
CNN	CNN-ORG	0.6963	0.4434	0.5347	0.4211	0.4753
	CNN-RND	0.8084	0.5349	0.7843	0.5680	0.6550
	CNN-CTGANSamp	0.8168	0.4298	**0.7988**	0.4536	0.6878
	CNN-Dice	0.4499	0.0900	0.2000	0.1241	0.0000
	CNN-Focal (This work)	**0.8677**	**0.6165**	0.6325	**0.5853**	**0.7606**

doubles kernel size at each iteration. There are 64 units in the first Bi-LSTM layer, 128 units in the second, and 128 units in the final Bi-LSTM layer. An activation function with softmax is used in the final layer, which is a dense layer that is fully connected. We evaluated CNN-BiLSTM using the open-source code provided at the CNN-BiLSTM repository [80].

PB-DID. PB-DID [81] uses an auxiliary dataset that shares many of the features of samples in the main dataset. So, we first trained PB-DID on the two datasets, Bot-IoT and WUSTL-IIoT-2021, since these two datasets share six common features. Bot-IoT and WUSTL-IIoT-2021 datasets have three common class types, namely Normal, DoS, and Reconnaissance. We added all the samples of these three common class types from WUSTL-IIoT-2021 dataset to Bot-IoT dataset. To train the model, this merged dataset has been used, and to test the model, the original Bot-IoT testing set has been used. Then, we trained PB-DID on WUSTL-IIoT-2021 and WUSTL-EHMS-2020 datasets in the second experiment; samples in these two datasets have twelve common features. The WUSTL-IIoT-2021 and WUSTL-EHMS-2020 datasets have been processed using a similar approach. We combined samples in the attack class of WUSTL-IIoT-2021 dataset with the samples in the attack class of WUSTL-EHMS-2020 dataset since WUSTL-EHMS-2020 contains only two class types, namely, Normal and Attack. We followed the same procedure for normal class. We used the open-source code provided for PB-DID [81] to evaluate its performance.

5. Results of our performance evaluation

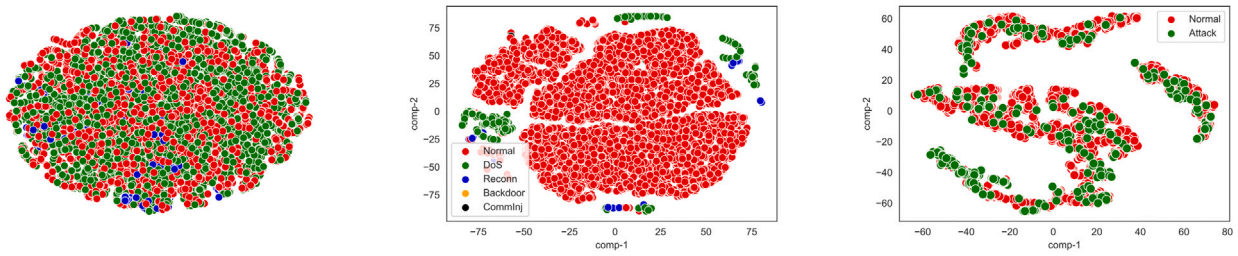
First, in Section 5.1, we present a quantitative analysis of the performance of various approaches on each dataset with respect to the metrics given in Table 1. Then, we present a qualitative analysis of the results of three of the approaches by taking a closer look at their classification of a subset of randomly selected samples from one of the datasets and present a peer to peer comparison in Section 5.2.

5.1. Quantitative analysis

5.1.1. Analysis of performance of various approaches on Bot-IoT dataset.

Table 5 contains the performance results of various models we evaluated using the Bot-IoT dataset. FNN-Focal and CNN-Focal perform significantly better than competing methods with respect to accuracy, precision, F_1 score, and MCC score. Specifically, with respect to the MCC score, FNN-Focal and CNN-Focal show an improvement of 10 and 29 percentage points over FNN-ORG and CNN-ORG, respectively. Additionally, with respect to MCC score, FNN-Focal and CNN-Focal perform the best among all the compared methods; for example, with respect to the MCC score, FNN-Focal shows an improvement of 12 percentage points over FNN-RND and CNN-Focal shows an improvement of 8 percentage points CNN-CTGANSamp. Similarly, both FNN-Focal and CNN-Focal show improvements in accuracy by 3 and 17 percentage points over FNN-ORG and CNN-ORG, respectively; following the similar trend, with respect to F_1-score, FNN-Focal and CNN-Focal show improvements of 3 and 16 percentage points respectively. This performance gain over competing models can be attributed to the dynamically scaled gradient updates enabled by focal loss in both FNN-Focal and CNN-Focal. To recall about the class imbalance issues in the test set as well, we present the t-Distributed Stochastic Neighboring Entities (t-SNE) [82] distribution of Bot-IoT test set in Fig. 4(a) where DDoS and DoS constitute the majority of the testing set. To determine the t-SNE projection of the test set, we take 10% of samples from each type of class.

Further analyzing the performance of the competing models, we notice that FNN-RND, FNN-Dice, and CNN-Dice have significantly lower F_1-scores than FNN-ORG and CNN-ORG. Similarly, compared to FNN-ORG, accuracy of FNN-RND has decreased by 2 percentage points, while accuracy of CTGANSamp also slightly decreased compare to FNN-ORG. Upon digging deep into the models that employ dice loss function to overcome the class imbalance issue, we observe that dice coefficient becomes very low, which results in a high loss and, as a result, models being trained using dice loss are not performing well in minority classes. Specifically, FNN-Dice and CNN-Dice both perform poorly with accuracy of only 45%. These results highlight that random oversampling or even dice loss function cannot overcome the class imbalance issue in a robust manner. It is critical to highlight that the two most recent state-of-the-art models CNN-BiLSTM and PB-DID both have poor accuracy of 14% and 52%, respectively. In terms of F_1-score, both state-of-the-art models perform poorly as they achieve score of 7% and 14% respectively. FNN-CTGANSamp shows an improvement



(a) Bot-IoT (b) WUSTL-IIoT-2021 (c) WUSTL-EHMS-2020

Fig. 4. t-SNE visualization of samples for the Bot-IoT, WUSTL-IIoT-2021, and WUSTL-EHMS-2020 testing sets.

Table 6
Performance of the evaluated methods on WUSTL-IIoT-2021.

DL models	Classifier's name	Acc	Pre	Rec	F_1	MCC
State-of-the-art	CNN-BiLSTM	0.9624	0.7222	0.4349	0.5086	0.6829
	PB-DID	0.0356	0.2105	0.1110	0.0214	−0.6265
FNN	FNN-ORG	0.9620	0.7124	0.4338	0.5040	0.6798
	FNN-RND	0.5805	0.4243	**0.7708**	0.4185	0.3440
	FNN-CTGANSamp	0.7342	0.5953	0.5848	0.5057	0.3773
	FNN-Dice	0.9271	0.1854	0.2000	0.1924	0.0000
	FNN-Focal (This work)	**0.9895**	**0.7722**	0.6406	**0.6848**	**0.9232**
CNN	CNN-ORG	0.9799	0.7486	0.6142	0.6558	0.8596
	CNN-RND	0.9635	0.6059	**0.8126**	0.5800	0.7738
	CNN-CTGANSamp	0.9810	0.7939	0.6608	0.6940	0.8714
	CNN-Dice	0.9271	0.1854	0.2000	0.1924	0.0000
	CNN-Focal (This work)	**0.9821**	**0.8854**	0.6651	**0.7050**	**0.8792**

over FNN-ORG, however it is important to observe that its performance is still not as good as that of FNN-Focal. Performance of CNN-RND and CNN-CTGANSamp have also improved over the trivial baseline, but not as much as CNN-Focal.

Finally, we notice that FNN-CTGANSamp and CNN-CTGANSamp perform the best with respect to recall metric that highlights that adding synthetic data has the potential to improve the recall. But, interestingly, the same models show significantly poor performance with respect to precision. This result leads to the conclusion that CTGANSamp is improving recall at the expense of precision. To summarize the results, we conclude that the proposed approach using focal loss function consistently outperforms all the competing models with respect to the robust metrics (e.g., F_1-score and MCC) on the Bot-IoT dataset and is able to find the sweet spot for precision and recall, thus improving the overall F_1-score and MCC score.

5.1.2. Analysis of performance of various approaches on WUSTL-IIoT-2021 dataset.

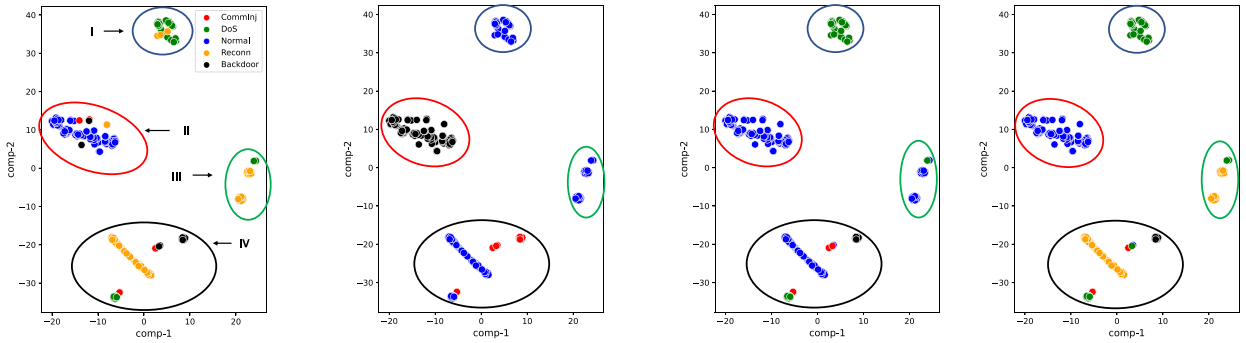
Fig. 4(b), shows the t-SNE projection of 10% of the samples of each class in the WUSTL-IIoT-2021 testing dataset. In this dataset, majority of samples are of normal class type. There are very few samples of type backdoors and command injections. Table 6 contains the performance results of different DL classifiers and the state-of-the-art methods on WUSTL-IIoT-2021 dataset. In general, performance of all the approaches on this dataset is similar to their performance on Bot-IoT dataset. FNN-Focal and CNN-Focal have highest accuracy, precision, F_1 score, and MCC score among their peers. MCC score of FNN-Focal is 25 percentage points higher than that of FNN-ORG, the nearest competitor among all FNN-based methods; even though MCC score of CNN-Focal is only 2 percentage points higher than that of CNN-ORG, it has the highest MCC score among all CNN-based methods. F_1-scores of FNN-Focal and CNN-Focal have improved by 18 and 5 percentage points over that of FNN-ORG and CNN-ORG respectively. With respect to accuracy, FNN-Focal and CNN-Focal show an improvement of 3% and 1% over FNN-ORG and CNN-ORG respectively.

With respect to accuracy, FNN-RND, CNN-RND, FNN-CTGANSamp, PB-DID, FNN-Dice, and CNN-Dice all perform worse than FNN-ORG and CNN-ORG. Compared to FNN-ORG and CNN-ORG, F_1-scores of FNN-RND, CNN-RND, FNN-Dice, and CNN-Dice have declined significantly. FNN-CTGANSamp performs only marginally better than the FNN-ORG with respect to F_1-score. FNN-Focal shows 18 percentage points improvement over FNN-CTGANSamp and 6 percentage points improvement over FNN-ORG with respect to precision. FNN-RND, FNN-CTGANSamp, and FNN-Dice have much lower MCC scores compared to FNN-ORG. Similarly, CNN-RND, and CNN-Dice have significantly lower MCC scores compared to CNN-ORG. Furthermore, both state-of-the-art models have poor MCC scores, with CNN-BiLSTM having 68% and PB-DID having negative 62%.

We observe that FNN-RND and CNN-RND perform the best with respect to recall at the expense of precision metric, since they are among the worst on the precision metric. Moreover, it is interesting to note that the same models perform significantly worse than other models when it comes to F_1-score and MCC score. Our conclusion based on the performance on the WUSTL-IIOT-2021 dataset is that our proposed method (i.e., DL using focal loss) consistently outperforms all competing models with respect to robust metrics (e.g., F_1-score and MCC) as well as finds better spot with respect to precision and recall.

Table 7
Performance of the evaluated methods on WUSTL-EHMS-2020.

DL models	Classifier's name	Acc	Pre	Rec	F_1	MCC
State-of-the-art	CNN-BiLSTM	0.9250	0.9010	0.7305	0.7851	0.6080
	PB-DID	0.8741	0.4372	0.4998	0.4664	−0.0066
FNN	FNN-ORG	0.9308	0.9382	0.7359	0.7975	0.6430
	FNN-RND	0.9305	0.9339	0.7367	0.7974	0.6410
	FNN-CTGANSamp	0.9299	0.9294	0.7364	0.7962	0.6372
	FNN-Dice	0.9302	0.9336	0.5000	0.4665	0.0000
	FNN-Focal (This work)	**0.9326**	**0.9524**	**0.7369**	**0.8011**	**0.6548**
CNN	CNN-ORG	0.9289	0.9284	0.7327	0.7927	0.6316
	CNN-RND	0.9296	0.9272	**0.7362**	0.7956	0.6354
	CNN-CTGANSamp	0.9274	0.9107	0.7360	0.7921	0.6227
	CNN-Dice	0.1256	0.0628	0.5000	0.1116	0.0000
	CNN-Focal (This work)	**0.9308**	**0.9423**	0.7338	**0.7963**	**0.6431**



(a) Ground Truth (b) FNN-ORG (c) CNN-BiLSTM (d) FNN-Focal

Fig. 5. Qualitative analysis of FNN-ORG, CNN-BiLSTM, and FNN-Focal on WUSTL-IIoT-2021 dataset.

5.1.3. Analysis of performance of various approaches on WUSTL-EHMS-2020 dataset.

A similar performance trend is presented in Table 7 for WUSTL-EHMS-2020 dataset. Fig. 4(c), shows the t-SNE projection in the WUSTL-EHMS-2020 testing dataset. Both FNN-Focal and CNN-Focal perform better than baseline DL classifiers using traditional loss functions in terms of accuracy, precision, F_1 score, and MCC score. With respect to accuracy, precision, recall, F_1-score, and MCC score, performance of FNN-Focal has improved by $0.002, 2, 0.001, 0.003$, and 1.1 percentage points respectively, compared to its nearest competitor FNN-ORG. Similarly, With respect to accuracy, precision, F_1-score, and MCC score, performance of CNN-Focal has improved by $0.019, 2, 0.004$, and 0.012 percentage points respectively, compared to its nearest competitor CNN-ORG. So, both FNN-FocaL and CNN-Focal have shown only marginal improvement.

Compared to the baseline classifiers (FNN-ORG and CNN-ORG), performance of FNN-RND, CNN-RND, FNN-CTGANSamp, CNN-CTGANSamp, FNN-Dice, CNN-Dice, CNN-BiLSTM, and PB-DID have declined with respect to all metrics except recall; with respect to recall, CNN-RND shows slight improvement in performance over CNN-ORG. Compared to FNN-ORG, (i) F_1-score of FNN-CTGANSamp has decreased by 1 percentage point; (ii)F_1-score of FNN-Dice has decreased by 33 percentage points; (iii) F_1-score of FNN-RND has also seen a slight decline. MCC scores of FNN-ORG, FNN-RND and FNN-CTGANSamp differ by at most 1 percentage points; MCC score of CNN-Focal has increased by 2 percentage points and 4 percentage points respectively compared to the MCC scores of CNN-CTGANSamp and CNN-BiLSTM; MCC score of PB-DID is in the negative. With respect to recall, CNN-RND performed only slightly (by 0.0024 percentage points) better than CNN-Focal. All scores of FNN-Focal are better compared to all competing models. *We conclude from these results that the proposed method using focal loss function consistently outperformed all competing models with respect to robust metrics* F_1*-score and MCC and also finds better spot with respect to accuracy, precision and recall. On all the datasets, MCC score for FNN-Dice and CNN-Dice is zero. This is due to the fact that dice loss does not perform well on minority classes.*

5.2. Qualitative analysis

We drew 400 samples from WUSTL-IIoT-2021 testing set and performed t-SNE projection on how selected methods performed on this sample. In order to perform the quantitative analysis, we chose FNN-ORG, CNN-BiLSTM, and FNN-Focal which have accuracy of 96.20, 96.24, and 98.95, respectively on the dataset WUSTL-IIoT-2021. The ground truth (i.e., data samples in each of the five attack classes are marked correctly using colors red, green, blue, yellow and black) of the selected samples is shown in Fig. 5(a). We grouped the selected samples into four groups (note that this grouping is not based on the attack class types). We encircled these four groups in four different colors: (i) blue, (ii) red, (iii) green, and (iv) black.

Most of the samples inside the blue circle of ground truth, are of attack type DoS and a few of these samples are of type Reconnaissance. Under FNN-ORG, all the samples inside the blue circle have been misclassified as Normal; on the other hand, CNN-BiLSTM and FNN-Focal classify them all as DoS, misclassifying the few Reconnaissance as DoS. A vast majority of samples enclosed in the red circle in the ground truth are Normal, while some are Command Injection, Reconnaissance and Backdoor samples; under FNN-ORG, all the samples within the red circle are misclassified as Backdoors; however, CNN-BiLSTM and FNN-Focal classify all of them as normal, misclassifying the few Command Injection, Reconnaissance and Backdoor samples as normal.

Most of the samples enclosed inside the green circle of ground truth (Fig. 5(a)) are Reconnaissance, while some are DoS. Fig. 5(b) and (c) show that the samples inside the green circle are not classified correctly by FNN-ORG and CNN-BiLSTM; on the other hand, these samples are classified correctly by FNN-Focal, as shown in Fig. 5(d).

Samples enclosed inside the black circle of ground truth (Fig. 5(a)) are of attack types Reconnaissance, Command Injection, Backdoor, and DoS. FNN-ORG (Fig. 5(b)) misclassified all the Reconnaissance as Normal, all the Backdoor as Command Injection, all the DoS as Normal; CNN-BiLSTM (Fig. 5(c)) also misclassifies these samples in the same way; in contrast, FNN-Focal(Fig. 5(d)) appears to classify all samples correctly, except for a few Backdoor attacks being misclassified as DoS attacks.

To summarize, t-SNE projection of performance various approaches on samples selected from WUSTL-IIoT-2021 testing set also confirms that FNN-Focal performs better than FNN-ORG and CNN-BiLSTM.

5.3. Limitations and future work

As with any intrusion detection system, the proposed intrusion detection approach is not perfect and might have several known or unknown (to the authors) limitations. In the following, we describe the limitations of our proposed approach and relevant precautions. First of all, like any supervised machine learning-based system, the proposed approach has been trained using the specified publicly available datasets, tested on the well-defined test sets only, and the reported results are accurate as per our experiments. However, it is to be noted that the performance of the proposed approach might not be similar in the real-world deployments, since the proposed system has not been tested for "In the Wild" setting. Nonetheless, we want to emphasize that the proposed approach provides basis for building systems that might be deployed in the real-world. Other than that, as with any machine learning-based system, the proposed approach can be used to train an adversarial neural network to launch an attack on any machine learning based intrusion detection system. For example, a Generative Adversarial Networks (GANs)-based approach can be used to train such an adversarial neural network to launch an attack where our proposed system (or any similar approach) may serve as a discriminator. Last but not least, we observe that although our proposed approach outperforms state-of-the-art intrusion detection systems including many strong baseline models, it is far from being sufficient to be deployable in real-world. It highlights that there is need for more research in such an important research area and robust ML-based intrusion detection systems are needed that may be deployed in real-world without any manual and laborious handcrafting.

6. Conclusion

Over the past decade, researchers have used ML techniques to design and implement intrusion detection systems for computer networks, in general; but not much work has been done in the area of intrusion detection for IoT. In this paper, we addressed this gap and presented a novel DL-based intrusion detection method using focal loss for IoT and evaluated its performance using three different datasets in three different domains of IoT. We compared the performance of our approach with several baseline models as well as state-of-the art models through extensive experimental evaluation. It is noteworthy to mention that our evaluation shows that our approach (CNN-Focal) performed better with respect to accuracy, precision, F_1 score and MCC score by as much as **24%**, **39%**, **39%**, and **60%**, respectively, compared to baseline model CNN-ORG over the Bot-IoT dataset. Its performance was also much better than some of the state-of-the art approaches that we compared.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] Ayesha S. Dina, D. Manivannan, Intrusion detection based on machine learning techniques in computer networks, Internet of Things (ISSN: 2542-6605) 16 (2021) 100462, http://dx.doi.org/10.1016/j.iot.2021.100462, URL https://www.sciencedirect.com/science/article/pii/S2542660521001037.

[2] Emilie Bout, Valeria Loscri, Antoine Gallais, How machine learning changes the nature of cyberattacks on IoT networks: A survey, IEEE Commun. Surv. Tutor. 24 (1) (2022) 248–279, http://dx.doi.org/10.1109/COMST.2021.3127267.

[3] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, BeyoundTrust, 2016.

[4] Li Li, Amir Ghasemi, IoT-enabled machine learning for an algorithmic spectrum decision process, IEEE Internet Things J. 6 (2) (2019) 1911–1919, http://dx.doi.org/10.1109/JIOT.2018.2883490.

[5] Yang Xin, Lingshuang Kong, Zhi Liu, Yuling Chen, Yanmiao Li, Hongliang Zhu, Mingcheng Gao, Haixia Hou, Chunhua Wang, Machine learning and deep learning methods for cybersecurity, IEEE Access 6 (2018) 35365–35381.

[6] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, Kalyan Veeramachaneni, Modeling tabular data using conditional GAN, 2019, arXiv preprint arXiv:1907.00503.

[7] Ayesha Siddiqua Dina, A.B. Siddique, D. Manivannan, Effect of balancing data using synthetic data on the performance of machine learning classifiers for intrusion detection in computer networks, IEEE Access 10 (2022) 96731–96747, http://dx.doi.org/10.1109/ACCESS.2022.3205337.

[8] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, Piotr Dollár, Focal loss for dense object detection, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 2980–2988.

[9] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, Benjamin Turnbull, Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset, Future Gener. Comput. Syst. 100 (2019) 779–796.

[10] M. Zolanvari, M.A. Teixeira, L. Gupta, K.M. Khan, R. Jain, WUSTL-IIOT-2021 Dataset for IIoT cybersecurity research, Washington University in St. Louis, USA, October 2021, 2021, https://www.cse.wustl.edu/~jain/iiot2/index.html.

[11] A.A. Hady, A. Ghubaish, T. Salman, D. Unal, R. Jain, Intrusion detection system for healthcare systems using medical and network data: A comparison study, IEEE Access (2020) 106576–106584, URL https://www.cse.wustl.edu/~jain/ehms/index.html.

[12] Carole H Sudre, Wenqi Li, Tom Vercauteren, Sebastien Ourselin, M Jorge Cardoso, Generalised dice overlap as a deep learning loss function for highly unbalanced segmentations, in: Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support, Springer, 2017, pp. 240–248.

[13] Chen Shen, Holger R Roth, Hirohisa Oda, Masahiro Oda, Yuichiro Hayashi, Kazunari Misawa, Kensaku Mori, On the influence of dice loss function in multi-class organ segmentation of abdominal CT using 3D fully convolutional networks, 2018, arXiv preprint arXiv:1801.05912.

[14] Jay Sinha, M. Manollas, Efficient deep CNN-BiLSTM model for network intrusion detection, in: Proceedings of the 2020 3rd International Conference on Artificial Intelligence and Pattern Recognition, 2020, pp. 223–231.

[15] Muhammad Zeeshan, Qaiser Riaz, Muhammad Ahmad Bilal, Muhammad K Shahzad, Hajira Jabeen, Syed Ali Haider, Azizur Rahim, Protocol-based deep intrusion detection for DoS and DDoS attacks using UNSW-NB15 and Bot-IoT data-sets, IEEE Access 10 (2021) 2269–2283.

[16] Imtiaz Ullah, Qusay H. Mahmoud, Design and development of a deep learning-based model for anomaly detection in IoT networks, IEEE Access 9 (2021) 103906–103926, http://dx.doi.org/10.1109/ACCESS.2021.3094024.

[17] Abdallah R. Gad, Ahmed A. Nashat, Tamer M. Barkat, Intrusion detection system using machine learning for vehicular Ad Hoc networks based on ToN-IoT dataset, IEEE Access 9 (2021) 142206–142217, http://dx.doi.org/10.1109/ACCESS.2021.3120626.

[18] Nour Moustafa, A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets, Sustainable Cities Soc. (ISSN: 2210-6707) 72 (2021) 102994, http://dx.doi.org/10.1016/j.scs.2021.102994, URL https://www.sciencedirect.com/science/article/pii/S2210670721002808.

[19] Zhipeng Liu, Niraj Thapa, Addison Shaver, Kaushik Roy, Xiaohong Yuan, Sajad Khorsandroo, Anomaly detection on IoT network intrusion using machine learning, in: Proceedings of 2020 International Conference on Artificial Intelligence, Big Data, Computing and Data Communication Systems (IcABCD), 2020, pp. 1–5, http://dx.doi.org/10.1109/icABCD49160.2020.9183842.

[20] Alexander Branitskiy, Igor Kotenko, Igor Saenko, Applying machine learning and parallel data processing for attack detection in IoT, IEEE Trans. Emerg. Top. Comput. 9 (4) (2021) 1642–1653, http://dx.doi.org/10.1109/TETC.2020.3006351.

[21] Muhammad Shafiq, Zhihong Tian, Ali Kashif Bashir, Xiaojiang Du, Mohsen Guizani, CorrAUC: A malicious Bot-IoT traffic detection method in IoT network using machine-learning techniques, IEEE Internet Things J. 8 (5) (2021) 3242–3254, http://dx.doi.org/10.1109/JIOT.2020.3002255.

[22] Andrew Churcher, Rehmat Ullah, Jawad Ahmad, Sadaqat ur Rehman, Fawad Masood, Mandar Gogate, Fehaid Alqahtani, Boubakr Nour, William J. Buchanan, An experimental analysis of attack classification using machine learning in IoT networks, Sensors 21 (2021) 446, http://dx.doi.org/10.3390/s21020446.

[23] R.A. Disha, S. Waheed, Performance analysis of machine learning models for intrusion detection system using Gini Impurity-based Weighted Random Forest (GIWRF) feature selection technique, Cybersecurity 5 (2022) http://dx.doi.org/10.1186/s42400-021-00103-8.

[24] Nour Moustafa, Jill Slay, UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: Proceedings of 2015 Military Communications and Information Systems Conference (MilCIS), IEEE, 2015, pp. 1–6.

[25] Mahmudul Hasan, Md. Milon Islam, Md Ishrak Islam Zarif, M.M.A. Hashem, Attack and anomaly detection in IoT sensors in IoT sites using machine learning approaches, Internet of Things (ISSN: 2542-6605) 7 (2019) 100059, http://dx.doi.org/10.1016/j.iot.2019.100059, URL https://www.sciencedirect.com/science/article/pii/S2542660519300241.

[26] Abhishek Verma, Virender Ranga, Machine learning based intrusion detection systems for IoT applications, Wirel. Pers. Commun. 111 (2020) 2287–2310.

[27] Markus Ring, Sarah Wunderlich, Dominik Grüdl, Dieter Landes, Andreas Hotho, Creation of flow-based data sets for intrusion detection, J. Inform. Warfare 16 (4) (2017) 41–54.

[28] dataset link, NSL-KDD dataset, 2009, Available on http://nsl.cs.unb.ca/KDD/NSL-KDD.html.

[29] Kai Yang, Yuanming Shi, Yong Zhou, Zhanpeng Yang, Liqun Fu, Wei Chen, Federated machine learning for intelligent IoT via reconfigurable intelligent surface, IEEE Netw. 34 (5) (2020) 16–22, http://dx.doi.org/10.1109/MNET.011.2000045.

[30] Miloud Bagaa, Tarik Taleb, Jorge Bernal Bernabe, Antonio Skarmeta, A machine learning security framework for IoT systems, IEEE Access 8 (2020) 114066–114077, http://dx.doi.org/10.1109/ACCESS.2020.2996214.

[31] Pathum Chamikara Mahawaga Arachchige, Peter Bertok, Ibrahim Khalil, Dongxi Liu, Seyit Camtepe, Mohammed Atiquzzaman, A trustworthy privacy preserving framework for machine learning in industrial IoT systems, IEEE Trans. Ind. Inform. 16 (9) (2020) 6092–6102, http://dx.doi.org/10.1109/TII.2020.2974555.

[32] Liang Liu, Xiangyu Xu, Yulei Liu, Zuchao Ma, Jianfei Peng, A detection framework against CPMA attack based on trust evaluation and machine learning in IoT network, IEEE Internet Things J. 8 (20) (2021) 15249–15258, http://dx.doi.org/10.1109/JIOT.2020.3047642.

[33] Aaisha Makkar, Sahil Garg, Neeraj Kumar, M. Shamim Hossain, Ahmed Ghoneim, Mubarak Alrashoud, An efficient spam detection technique for IoT devices using machine learning, IEEE Trans. Ind. Inform. 17 (2) (2021) 903–912, http://dx.doi.org/10.1109/TII.2020.2968927.

[34] Souradip Roy, Juan Li, Yan Bai, A two-layer fog-cloud intrusion detection model for IoT networks, Internet of Things (ISSN: 2542-6605) 19 (2022) 100557, https://nam04.safelinks.protection.outlook.com/.

[35] Fan Liang, William Grant Hatcher, Weixian Liao, Weichao Gao, Wei Yu, Machine learning for security and the Internet of Things: The Good, the Bad, and the Ugly, IEEE Access 7 (2019) 158126–158147, http://dx.doi.org/10.1109/ACCESS.2019.2948912.

[36] Peiyuan Sun, Jianxin Li, Md Zakirul Alam Bhuiyan, Lihong Wang, Bo Li, Modeling and clustering attacker activities in IoT through machine learning techniques, Inform. Sci. (ISSN: 0020-0255) 479 (2019) 456–471, http://dx.doi.org/10.1016/j.ins.2018.04.065, URL https://www.sciencedirect.com/science/article/pii/S0020025518303311.

[37] Amar Amouri, Vishwa T. Alaparthy, Salvatore D. Morgera, A machine learning based intrusion detection system for Mobile Internet of Things, Sensors 20 (2020) 461, http://dx.doi.org/10.3390/s20020461.

[38] Arunan Sivanathan, Hassan Habibi Gharakheili, Vijay Sivaraman, Managing IoT cyber-security using programmable telemetry and machine learning, IEEE Trans. Netw. Serv. Manag. 17 (1) (2020) 60–74, http://dx.doi.org/10.1109/TNSM.2020.2971213.

[39] Mengyao Zheng, Dixing Xu, Linshan Jiang, Chaojie Gu, Rui Tan, Peng Cheng, Challenges of privacy-preserving machine learning in IoT, in: Proceedings of the First International Workshop on Challenges in Artificial Intelligence and Machine Learning for Internet of Things, ACM, 2019, pp. 1–7, http://dx.doi.org/10.1145/3363347.3363357.

[40] Tanujay Saha, Najwa Aaraj, Neel Ajjarapu, Niraj K. Jha, SHARKS: Smart hacking approaches for risk scanning in Internet-of-Things and cyber-physical systems based on machine learning, IEEE Trans. Emerg. Top. Comput. 10 (2) (2022) 870–885, http://dx.doi.org/10.1109/TETC.2021.3050733.

[41] Jorge Luis Guerra, Carlos Catania, Eduardo Veas, Datasets are not enough: Challenges in labeling network traffic, Comput. Secur. (ISSN: 0167-4048) 120 (2022) 102810, http://dx.doi.org/10.1016/j.cose.2022.102810, URL https://www.sciencedirect.com/science/article/pii/S0167404822002048.

[42] Omar Abdel Wahab, Intrusion detection in the IoT under data and concept drifts: Online deep learning approach, IEEE Internet Things J. (2022) 1, http://dx.doi.org/10.1109/JIOT.2022.3167005.

[43] Izhar Ahmed Khan, Nour Moustafa, Imran Razzak, M. Tanveer, Dechang Pi, Yue Pan, Bakht Sher Ali, XSRU-IoMT: Explainable simple recurrent units for threat detection in Internet of Medical Things networks, Future Gener. Comput. Syst. (ISSN: 0167-739X) 127 (2022) 181–193, http://dx.doi.org/10.1016/j.future.2021.09.010, URL https://www.sciencedirect.com/science/article/pii/S0167739X21003563.

[44] Mohamed Amine Ferrag, Othmane Friha, Leandros Maglaras, Helge Janicke, Lei Shu, Federated deep learning for cyber security in the Internet of Things: concepts, applications, and experimental analysis, IEEE Access 9 (2021) 138509–138542, http://dx.doi.org/10.1109/ACCESS.2021.3118642.

[45] Maede Zolanvari, Marcio A. Teixeira, Lav Gupta, Khaled M. Khan, Raj Jain, Machine learning-based network vulnerability analysis of Industrial Internet of Things, IEEE Internet Things J. 6 (4) (2019) 6822–6834, http://dx.doi.org/10.1109/JIOT.2019.2912022.

[46] Diana Yacchirema, Jara Suarez de Puga, Carlos Palau, Manuel Esteve, Fall detection system for elderly people using IoT and ensemble machine learning algorithm, Pers. Ubiquitous Comput. 23 (2019) 801–817, http://dx.doi.org/10.1007/s00779-018-01196-8.

[47] Gregory B. Rehm, Sang Hoon Woo, Xin Luigi Chen, Brooks T. Kuhn, Irene Cortes-Puch, Nicholas R. Anderson, Jason Y. Adams, Chen-Nee Chuah, Leveraging IoTs and machine learning for patient diagnosis and ventilation management in the intensive care unit, IEEE Pervasive Comput. 19 (3) (2020) 68–78, http://dx.doi.org/10.1109/MPRV.2020.2986767.

[48] Yair Meidan, Michael Bohadana, Asaf Shabtai, Martin Ochoa, Nils Ole Tippenhauer, Juan Davis Guarnizo, Yuval Elovici, Detection of unauthorized IoT devices using machine learning techniques, 2017, http://dx.doi.org/10.48550/ARXIV.1709.04647, URL https://arxiv.org/abs/1709.04647.

[49] Ola Salman, Imad H. Elhajj, Ali Chehab, Ayman Kayssi, A machine learning based framework for IoT device identification and abnormal traffic detection, Trans. Emerg. Telecommun. Technol. (2019) http://dx.doi.org/10.1002/ett.3743.

[50] Yongxin Liu, Jian Wang, Jianqiang Li, Shuteng Niu, Houbing Song, Machine learning for the detection and identification of Internet of Things Devices: A Survey, IEEE Internet Things J. 9 (1) (2022) 298–320, http://dx.doi.org/10.1109/JIOT.2021.3099028.

[51] W. Ma, X. Wang, M. Hu, Q. Zhou, Machine learning empowered trust evaluation method for IoT devices, IEEE Access 9 (2021) 65066–65077, http://dx.doi.org/10.1109/ACCESS.2021.3076118.

[52] Justin M. Beaver, Raymond C. Borges-Hink, Mark A. Buckner, An evaluation of machine learning methods to detect malicious SCADA communications, in: Proceedings of 2013 12th International Conference on Machine Learning and Applications, vol. 2, 2013, pp. 54–59, http://dx.doi.org/10.1109/ICMLA.2013.105.

[53] Anastasis Keliris, Hossein Salehghaffari, Brian Cairl, Prashanth Krishnamurthy, Michail Maniatakos, Farshad Khorrami, Machine learning-based defense against process-aware attacks on industrial control systems, in: Proceedings of 2016 IEEE International Test Conference, ITC, 2016, pp. 1–10, http://dx.doi.org/10.1109/TEST.2016.7805855.

[54] Oliver Eigner, Philipp Kreimel, Paul Tavolato, Detection of man-in-the-middle attacks on industrial control networks, in: Proceedings of 2016 International Conference on Software Security and Assurance, ICSSA, 2016, pp. 64–69, http://dx.doi.org/10.1109/ICSSA.2016.19.

[55] Imtiaz Ullah, Qusay H. Mahmoud, A hybrid model for anomaly-based intrusion detection in SCADA networks, in: Proceedings of 2017 IEEE International Conference on Big Data (Big Data), 2017, pp. 2160–2167, http://dx.doi.org/10.1109/BigData.2017.8258164.

[56] Youbiao He, Gihan J. Mendis, Jin Wei, Real-time detection of false data injection attacks in smart grid: A deep learning-based intelligent mechanism, IEEE Trans. Smart Grid 8 (5) (2017) 2505–2516, http://dx.doi.org/10.1109/TSG.2017.2703842.

[57] Sasanka Potluri, Navin Francis Henry, Christian Diedrich, Evaluation of hybrid deep learning techniques for ensuring security in networked control systems, in: Proceedings of 2017 22nd IEEE International Conference on Emerging Technologies and Factory Automation, ETFA, 2017, pp. 1–8, http://dx.doi.org/10.1109/ETFA.2017.8247662.

[58] Thiago Alves, Rishabh Das, Thomas Morris, Embedding encryption and machine learning intrusion prevention systems on programmable logic controllers, IEEE Embedded Syst. Lett. 10 (3) (2018) 99–102, http://dx.doi.org/10.1109/LES.2018.2823906.

[59] Mariam M.N. Aboelwafa, Karim G. Seddik, Mohamed H. Eldefrawy, Yasser Gadallah, Mikael Gidlund, A machine-learning-based technique for false data injection attacks detection in industrial IoT, IEEE Internet Things J. 7 (9) (2020) 8462–8471, http://dx.doi.org/10.1109/JIOT.2020.2991693.

[60] Saurav Chakraborty, Agnieszka Onuchowsk, Sagar Samtani, Wolfgang Jank a nd Brandon Wolfram, Machine learning for automated industrial IoT attack detection: An efficiency-complexity trade-off, ACM Trans. Manag. Inform. Syst. 12 (2021) 1–28, http://dx.doi.org/10.1145/3460822.

[61] H. Zhou, C. She, Y. Deng, M. Dohler, A. Nallanathan, Machine Learning for Massive Industrial Internet of Things, IEEE Wirel. Commun. 28 (4) (2021) 81–87, http://dx.doi.org/10.1109/MWC.301.2000478.

[62] Eric Gyamfi, Anca Delia Jurcut, Novel online network intrusion detection system for industrial IoT based on OI-SVDD and AS-ELM, IEEE Internet Things J. (2022) 1, http://dx.doi.org/10.1109/JIOT.2022.3172393.

[63] Izhar Ahmed Khan, Marwa Keshk, Dechang Pi, Nasrullah Khan, Yasir Hussain, Hatem Soliman, Enhancing IIoT networks protection: A robust security model for attack detection in Internet Industrial Control Systems, Ad Hoc Netw. (ISSN: 1570-8705) 134 (2022) 102930, http://dx.doi.org/10.1016/j.adhoc.2022.102930, URL https://www.sciencedirect.com/science/article/pii/S1570870522001159.

[64] Akbar Telikani, Jun Shen, Jie Yang, Peng Wang, Industrial IoT intrusion detection via evolutionary cost-sensitive learning and fog computing, IEEE Internet Things J. (2022) 1, http://dx.doi.org/10.1109/JIOT.2022.3188224.

[65] Nickolaos Koroniotis, Nour Moustafa, Francesco Schiliro, Praveen Gauravaram, Helge Janicke, The SAir-IIoT cyber testbed as a service: A novel cybertwins architecture in IIoT-based smart airports, IEEE Trans. Intell. Transp. Syst. (2021) 1–14, http://dx.doi.org/10.1109/TITS.2021.3106378.

[66] Michael Yeung, Evis Sala, Carola-Bibiane Schönlieb, Leonardo Rundo, Unified focal loss: Generalising dice and cross entropy-based losses to handle class imbalanced medical image segmentation, Comput. Med. Imaging Graph. 95 (2022) 102026.

[67] Ken CL Wong, Mehdi Moradi, Hui Tang, Tanveer Syeda-Mahmood, 3D segmentation with exponential logarithmic loss for highly unbalanced object sizes, in: International Conference on Medical Image Computing and Computer-Assisted Intervention, Springer, 2018, pp. 612–619.

[68] Aida Catic, Lejla Gurbeta, Amina Kurtovic-Kozaric, Senad Mehmedbasic, Almir Badnjevic, Application of neural networks for classification of Patau, Edwards, Down, Turner and Klinefelter Syndrome based on first trimester maternal serum screening data, ultrasonographic findings and patient demographics, BMC Med. Genom. 11 (1) (2018) 1–12.

[69] R. Arulmurugan, H. Anandakumar, Early detection of lung cancer using wavelet feature descriptor and feed forward back propagation neural networks classifier, in: Computational Vision and Bio Inspired Computing, Springer, 2018, pp. 103–110.

[70] Jie Yang, Jun Ma, Feed-forward neural network training using sparse representation, Expert Syst. Appl. 116 (2019) 255–264.

[71] Kaivan Kamali, Deep learning (part 1) - feedforward neural networks (FNN) (galaxy training materials), 2021, Online URL https://training.galaxyproject.org/training-material/topics/statistics/tutorials/FNN/tutorial.html (Accessed 16 March 2022)].

[72] Jinzhu Lu, Lijuan Tan, Huanyu Jiang, Review on convolutional neural network (CNN) applied to plant leaf disease classification, Agriculture 11 (8) (2021) 707.

[73] Guanghai Dai, Jun Zhou, Jiahui Huang, Ning Wang, HS-CNN: A CNN with hybrid convolution scale for EEG motor imagery classification, J. Neural Eng. 17 (1) (2020) 016025.

[74] Huy Phan, Fernando Andreotti, Navin Cooray, Oliver Y Chén, Maarten De Vos, Joint classification and prediction CNN framework for automatic sleep stage classification, IEEE Trans. Biomed. Eng. 66 (5) (2018) 1285–1296.
[75] Chunyan Yu, Rui Han, Meiping Song, Caiyu Liu, Chein-I Chang, A simplified 2D-3D CNN architecture for hyperspectral image classification based on spatial–spectral fusion, IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens. 13 (2020) 2485–2501.
[76] Youness Khourdifi, Mohamed Bahaj, Heart disease prediction and classification using machine learning algorithms optimized by particle swarm optimization and ant colony optimization, Int. J. Intell. Eng. Syst. 12 (1) (2019) 242–252.
[77] Thuy T.T. Nguyen, Grenville Armitage, A survey of techniques for internet traffic classification using machine learning, IEEE Commun. Surv. Tutor. 10 (4) (2008) 56–76.
[78] Davide Chicco, Giuseppe Jurman, The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation, BMC Genom. 21 (1) (2020) 1–13.
[79] Gaganpreet Sharma, Pros and cons of different sampling techniques, Int. J. Appl. Res. 3 (7) (2017) 749–752.
[80] code link, CNN-BiLSTM Repo. Available on: https://github.com/razor08/Efficient-CNN-BiLSTM-for-Network-IDS.
[81] code link, PB-DID Repo. Available at: https://github.com/nuttysunday/Protocol-Based-Deep-Intrusion-Detection-for-DoS-Normal-and-DDoS-Attacks.
[82] Laurens Van der Maaten, Geoffrey Hinton, Visualizing data using t-SNE, J. Mach. Learn. Res. 9 (11) (2008).